Exercício 8.3: Depuração: Job estourado, Secret inválido e Service sem endpoints

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1.
- Namespace `store-debug`, separado do `store-dev` para não misturar com o CronJob do 8.2 que rodava ao mesmo tempo.
- Sequência completa de comandos em `11-commands.sh`.

Cenário A: Job estourado

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: relatorio-quebrado
  namespace: store-debug
spec:
  backoffLimit: 2
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: relatorio
          image: busybox:1.37
          command: ["cat", "/dados/relatorio.csv"]
```

(i) Estado observado

```
$ kubectl -n store-debug get job,pods
NAME                           STATUS   COMPLETIONS   DURATION   AGE
job.batch/relatorio-quebrado   Failed   0/1           33s        33s

NAME                           READY   STATUS   RESTARTS   AGE
pod/relatorio-quebrado-6shtf   0/1     Error    0          33s
pod/relatorio-quebrado-p78hs   0/1     Error    0          3s
pod/relatorio-quebrado-w8rzw   0/1     Error    0          23s
```

Três Pods em `Error`: 1 tentativa + 2 retentativas (`backoffLimit: 2`). Com `restartPolicy: Never` cada retentativa é um Pod novo, e o intervalo entre eles cresce (10s, depois 20s) pelo back-off exponencial do controller.

(ii) Comando que revelou a causa

```
$ kubectl -n store-debug describe job relatorio-quebrado
Pods Statuses:    0 Active (0 Ready) / 0 Succeeded / 3 Failed
Events:
  Type     Reason                Age   From            Message
  ----     ------                ----  ----            -------
  Normal   SuccessfulCreate      33s   job-controller  Created pod: relatorio-quebrado-6shtf
  Normal   SuccessfulCreate      23s   job-controller  Created pod: relatorio-quebrado-w8rzw
  Normal   SuccessfulCreate      3s    job-controller  Created pod: relatorio-quebrado-p78hs
  Warning  BackoffLimitExceeded  0s    job-controller  Job has reached the specified backoff limit

$ kubectl -n store-debug get job relatorio-quebrado -o jsonpath='{range .status.conditions[*]}...'
FailureTarget=True reason=BackoffLimitExceeded msg=Job has reached the specified backoff limit
Failed=True reason=BackoffLimitExceeded msg=Job has reached the specified backoff limit

$ kubectl -n store-debug logs -l job-name=relatorio-quebrado --prefix
[pod/relatorio-quebrado-p78hs/relatorio] cat: can't open '/dados/relatorio.csv': No such file or
directory
[pod/relatorio-quebrado-w8rzw/relatorio] cat: can't open '/dados/relatorio.csv': No such file or
directory
[pod/relatorio-quebrado-6shtf/relatorio] cat: can't open '/dados/relatorio.csv': No such file or
directory
```

O `describe` mostra o motivo do Job (`BackoffLimitExceeded`), e o `logs` mostra o motivo de cada Pod: o arquivo não existe.

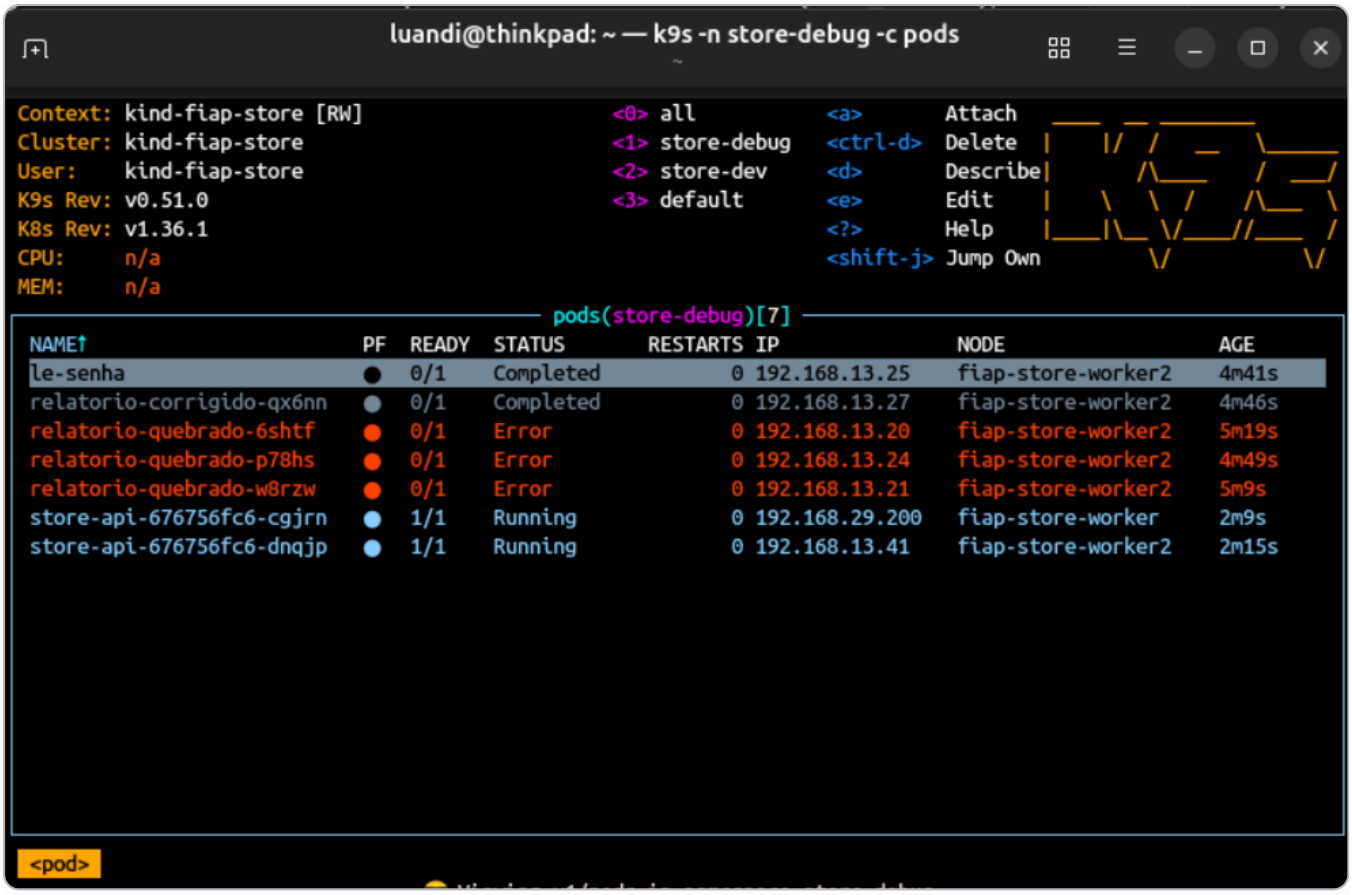
(iii) Correção: o arquivo passa a existir via ConfigMap montado em `/dados`. Um Job é imutável no `template`, então a correção é um Job novo.

```yaml
# correcao: o arquivo passa a existir (ConfigMap montado em /dados)
apiVersion: v1
kind: ConfigMap
metadata:
  name: relatorio
  namespace: store-debug
data:
  relatorio.csv: |
    pedido,valor
    1,99.90
---
apiVersion: batch/v1
kind: Job
metadata:
  name: relatorio-corrigido
  namespace: store-debug
spec:
  backoffLimit: 2
  template:
    spec:
      restartPolicy: Never
      containers:
        - name: relatorio
          image: busybox:1.37
          command: ["cat", "/dados/relatorio.csv"]
          volumeMounts:
            - name: dados
              mountPath: /dados
      volumes:
        - name: dados
          configMap:
            name: relatorio
```

(iv) Evidência da correção

```
$ kubectl -n store-debug get job relatorio-corrigido
NAME                 STATUS     COMPLETIONS   DURATION   AGE
relatorio-corrigido   Complete   1/1           3s         3s

$ kubectl -n store-debug logs job/relatorio-corrigido
pedido,valor
1,99.90
```



Cenário B: Secret inválido e Secret com quebra de linha

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
  namespace: store-debug
data:
  SENHA: senha@123
```

(i) Estado observado: o Secret não é criado.

(ii) Comando e mensagem exata

```
$ kubectl apply -f 04-secret-invalido.yaml
Error from server (BadRequest): error when creating "04-secret-invalido.yaml": Secret in version "v1"
cannot be handled as a Secret: illegal base64 data at input byte 5
```

O byte 5 (contando do zero) é o `@` de `senha@123`, fora do alfabeto base64. O erro vem do API server na decodificação, antes de qualquer validação do objeto.

Segundo Secret, com base64 válido gerado por `echo 'senha' | base64` (sem `-n`):

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
  namespace: store-debug
data:
  # echo senha | base64  (sem -n: inclui o \n)
  SENHA: c2VuaGEK
```

O Pod abaixo imprime o tamanho da variável e os bytes com `od -c`:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: le-senha
  namespace: store-debug
spec:
  restartPolicy: Never
  containers:
    - name: le-senha
      image: busybox:1.37
      command: ["sh", "-c", "echo tamanho=${#SENHA}; printf %s \"$SENHA\" | od -c"]
      env:
        - name: SENHA
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: SENHA
```

```
$ kubectl -n store-debug logs le-senha
tamanho=6
0000000   s   e   n   h   a  \n
0000006
```

`senha` tem 5 caracteres e o container recebeu 6: o sexto é o `\n` que o `echo` colocou antes do `base64`. `c2VuaGEK` termina em `K`, que é o `\n` codificado. Na prática, a senha do banco chega errada e o login falha sem nenhum erro de Kubernetes.

(iii) Correção: `echo -n senha | base64` (ou usar `stringData:`, que dispensa o base64).

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
  namespace: store-debug
data:
  # echo -n senha | base64
  SENHA: c2VuaGE=
```

(iv) Evidência da correção

```
$ kubectl -n store-debug logs le-senha
tamanho=5
0000000   s   e   n   h   a
0000005
```

Cenário C1: selector com erro de digitação

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
  namespace: store-debug
spec:
  replicas: 2
  selector:
    matchLabels:
      app: store-api
  template:
    metadata:
      labels:
        app: store-api
    spec:
      containers:
        - name: store-api
          image: store-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          readinessProbe:
            httpGet:
              path: /ready
              port: http
            periodSeconds: 5
```

```yaml
---
apiVersion: v1
kind: Service
metadata:
  name: store-api
  namespace: store-debug
spec:
  selector:
    app: storeapi # erro: sem hifen
  ports:
    - port: 80
      targetPort: http
```

(i) Estado observado: os Pods estão saudáveis, mas o Service não tem para onde mandar tráfego.

```
$ kubectl -n store-debug get pods -l app=store-api -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP               NODE
store-api-676756fc6-bv5t7    1/1     Running   0          7s    192.168.29.197   fiap-store-worker
store-api-676756fc6-xtplq    1/1     Running   0          7s    192.168.13.22    fiap-store-worker2
```

(ii) Comando e mensagem

```
$ kubectl -n store-debug get endpointslices -l kubernetes.io/service-name=store-api -o wide
NAME              ADDRESSTYPE   PORTS     ENDPOINTS   AGE
store-api-frtjk   IPv4          <unset>   <unset>     7s

$ kubectl -n store-debug describe svc store-api | grep -E "Selector|Endpoints"
Selector:                 app=storeapi
Endpoints:

$ kubectl -n store-debug run cliente --rm -i --image=curlimages/curl:8.10.1 --restart=Never -- \
    sh -c 'sleep 2; curl -sS -m 5 http://store-api/ready'
curl: (7) Failed to connect to store-api port 80 after 0 ms: Could not connect to server
 exit=7
```

O DNS resolve (é um ClusterIP válido), mas sem endpoints a conexão é recusada na hora. O `Selector: app=storeapi` contra a label `app: store-api` dos Pods mostra a causa.

(iii) Correção: `selector: app: store-api` (`09-store-api-corrigido.yaml`).

(iv) Evidência da correção

```
$ kubectl -n store-debug get endpointslices -l kubernetes.io/service-name=store-api -o wide
NAME              ADDRESSTYPE   PORTS   ENDPOINTS                     AGE
store-api-frtjk   IPv4          8000    192.168.13.22,192.168.29.197   24s

$ (mesmo cliente)
{"status":"ready"} exit=0
```

Cenário C2: selector certo, readinessProbe quebrada

Mesmo Deployment corrigido, com a probe apontando para `/pronto`:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: store-api
  namespace: store-debug
spec:
  replicas: 2
  selector:
    matchLabels:
      app: store-api
  template:
    metadata:
      labels:
        app: store-api
    spec:
      containers:
        - name: store-api
          image: store-api:1.0
          imagePullPolicy: IfNotPresent
          ports:
            - name: http
              containerPort: 8000
          readinessProbe:
            httpGet:
              path: /pronto # erro: rota inexistente
              port: http
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: store-api
  namespace: store-debug
```

```
spec:
  selector:
    app: store-api
  ports:
    - port: 80
      targetPort: http
```

Achado na primeira tentativa: aplicar a probe quebrada como *update* do Deployment saudável **não** derruba o serviço. Com 2 réplicas, o `maxUnavailable` padrão de 25% arredonda para 0, então o rollout cria 1 Pod novo `0/1`, para nele e mantém os 2 antigos atendendo. É o RollingUpdate protegendo contra a própria falha que o exercício quer provocar. Para reproduzir o cenário pedido, o Deployment foi apagado e criado já com a probe errada.

(i) Estado observado: Pods `Running`, mas `0/1`.

```
$ kubectl -n store-debug get pods -l app=store-api -o wide
NAME                         READY   STATUS    RESTARTS   AGE   IP               NODE
store-api-5f4695f6b7-46jl8   0/1     Running   0          30s   192.168.29.199   fiap-store-worker
store-api-5f4695f6b7-tvgh4   0/1     Running   0          30s   192.168.13.39    fiap-store-worker2
```

(ii) Comandos e mensagens

```
$ kubectl -n store-debug get endpointslices -l kubernetes.io/service-name=store-api \
    -o jsonpath='{range .items[*].endpoints[*]}{.addresses[0]} ready={.conditions.ready}{"\n"}{end}'
192.168.13.39 ready=false
192.168.29.199 ready=false

$ kubectl -n store-debug describe svc store-api | grep -E "^Endpoints"
Endpoints:

$ kubectl -n store-debug describe pod store-api-5f4695f6b7-46jl8
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       False
  ContainersReady             False
  PodScheduled                True
...
  Warning  Unhealthy  4s (x6 over 29s)   kubelet  spec.containers{store-api}: Readiness probe failed:
                                                  HTTP probe failed with statuscode: 404

$ (mesmo cliente)
curl: (7) Failed to connect to store-api port 80 after 10 ms: Could not connect to server
 exit=7
```

Diferença entre C1 e C2 no EndpointSlice: em C1 o slice não tem nenhum endereço. Em C2 os endereços estão lá, mas com `ready=false`. A coluna `ENDPOINTS` do `kubectl get endpointslices -o wide` **lista os dois IPs mesmo não prontos**, então ela sozinha não mostra o problema. O vazio aparece no `describe svc` (`Endpoints:` em branco), que só conta endereços prontos, e no campo `conditions.ready`. O kube-proxy segue a mesma regra e não roteia para endpoint `ready=false`, por isso o curl falha igual ao C1.

(iii) Correção: a probe volta para `/ready` (`09-store-api-corrigido.yaml`).

(iv) Evidência da correção

```
$ kubectl -n store-debug rollout status deploy/store-api
deployment "store-api" successfully rolled out

$ kubectl -n store-debug get endpointslices ... -o jsonpath='...'
192.168.29.199 ready=false     ← Pod antigo em Terminating
192.168.13.41 ready=true
192.168.29.200 ready=true

$ (mesmo cliente)
{"status":"ready"} exit=0
```

Critérios de aceite

Critério	Resultado
A: `reason: BackoffLimitExceeded` e exatamente 3 Pods `Error`	ok
B: mensagem `illegal base64 data`	ok (`at input byte 5`)
B: evidência numérica do caractere extra	ok (`tamanho=6` contra 5, `\n` no `od -c`)
C: endpoints vazios nas duas causas e populados após cada correção	ok, com a ressalva do C2 sobre a coluna `ENDPOINTS`